8장. A/B 테스트와 적응형 실험 설계

□ 이 장의 질문

○ 새 화면(B안)이 기존 화면(A안)보다 나은지 확인하려고 사용자를 반씩 나눠 2주 동안 실험한다. 그런데 실험이 끝날 때까지 절반의 사용자는 두 안 중 나쁜 쪽을 계속 본다. 이 손실이 아까워서, 중간에 결과를 들여다보고 좋아 보이면 멈추거나 잘되는 안에 사용자를 더 보내면 안 되는가

○ 이 장의 답은 “규칙 없이 하면 안 되고, 규칙을 정하면 된다”이다. 규칙 없이 중간에 멈추면 없는 효과를 있다고 보고할 확률이 커지고, 배분을 바꾸면 열등한 안의 효과를 정확히 재는 일을 내주게 된다.
무엇을 얻고 무엇을 내주는지를 숫자로 재는 방법을 배운다

□ 이 장을 읽는 방법

○ 이 장은 두 파트로 나뉜다

– 1부(8.1~8.2): 배분을 반반으로 고정한 A/B 테스트를 설계하는 계산(표본 크기·검정력)과, 중간에 결과를 들여다봐도 오류율이 부풀지 않게 하는 규칙(그룹 순차 설계)을 배운다

– 2부(8.3~8.7): 배분 비율 자체를 실험 도중에 바꾸는 설계(톰슨 샘플링·베이지안 적응형)를 돌려, 손실이 얼마나 줄고 대신 무엇을 포기하는지 잰다

○ 수식은 쓰지 않는다. 계산의 구조는 작은 숫자 예시로만 따라간다

○ 실습 데이터는 정답(참 전환율)을 알고 만든 연습용 데이터라서, 계산한 값이 참값에서 얼마나 빗나갔는지 채점할 수 있다

○ 실습 코드는 practice/chapter08/code/ 에 있다. 스크립트를 실행하면 콘솔 출력 전체가 practice/chapter08/results/<스크립트명>.log 로 저장되고, 그림도 results 폴더에 저장된다.
본문의 실행 결과 발췌는 모두 이 코드가 실제로 계산한 값이다

○ 용어 미리보기 — 본문에서 처음 나올 때마다 다시 풀어 설명하므로, 여기서는 눈에 익히기만 하면 된다

용어	쉬운 뜻
무작위 배정	누가 어느 안을 받을지를 동전 던지기처럼 우연으로만 정하는 것
A/B 테스트	사용자를 A안과 B안에 무작위로 나눠 성과를 비교하는 실험
전환율	그 안을 본 사람 100명 중 목표 행동(구매·가입)을 한 사람의 비율
유의수준(α)	효과가 없는데 “있다”고 잘못 판정하는 일을 허용하는 상한. 보통 5%

용어	쉬운 뜻
p값	효과가 없다고 가정할 때, 지금 같은 차이가 우연히 나올 확률
검정력	효과가 정말 있을 때 실험이 그 효과를 잡아낼 확률. 기준은 0.80
표본 크기	실험에 필요한 인원. 실험 시작 전에 계산해 문서에 적어 두는 숫자
중간 분석	실험이 끝나기 전에 결과를 미리 들여다보는 것
중단 경계	중간 분석에서 실험을 멈춰도 되는 기준선. 여러 번 보는 대가로 매번의 기준이 엄격해진다
톰슨 샘플링	성과가 좋아 보이는 안에 점점 더 많은 사용자를 보내는 배분 방법
사전분포·사후분포	데이터를 보기 전과 본 후에 “전환율이 얼마쯤일까”를 적은 확률의 분포
후회(regret)	처음부터 최적 안만 썼을 때와 비교해 잃은 성과의 양

1부. 고정 배분 A/B 테스트를 설계하고 종료 시점을 정한다

□ 1부 개요

○ 이 파트가 끝나면: 네 가지 효과 크기 시나리오의 필요 표본·검정력·실행 결과를 한 표로 읽고, 중간 분석이 1종 오류를 얼마나 부풀리는지(다섯 번 보면 0.05 → 0.1409)와 그 부풀림을 되돌리는 중단 경계를 판정에 쓸 수 있다

○ 다루는 절: 8.1 ~ 8.2

□ 8.1.1~8.1.2 실험이 기준점인 이유와 무작위 배정의 두 방식

○ 앞 장들의 방법은 모두 “관측 데이터에서 효과를 뽑으려면 무엇을 가정해야 하는가”에 기댄다. 도구변수는 배제제약을, 이중차분은 평행추세를 가정하며, 이 가정들은 데이터로 완전히 검증되지 않는다

– 그래서 결과를 읽는 사람은 늘 “그 가정이 맞다면”이라는 단서를 붙여야 한다

○ 실험은 이 단서를 없앤다. 처치를 받을지를 연구자가 동전 던지기로 정하는 것이 **무작위 배정**이며, 이렇게 하면 처치 여부가 나이·소득·측정하지 못한 동기와 모두 무관해진다

– 관측한 특성뿐 아니라 관측하지 못한 특성까지 평균적으로 비슷해지므로, 두 집단의 결과 평균 차이를 그대로 인과효과로 읽는다. 3장에서 매칭으로 애써 만들던 “비슷한 두 집단”이 설계 단계에서 이미 만들어진다

- 비즈니스에서는 이 절차를 **A/B 테스트**라고 부른다. 사용자를 A안(대조군)과 B안(처치군)에 무작위로 갈라 노출한 뒤 **전환율** — 그 안을 본 100명 중 목표 행동을 한 사람의 비율 — 을 비교하므로, 이름만 다를 뿐 의학의 무작위 대조시험(RCT)과 같은 구조다

- 무작위 배정에도 방식이 두 가지 있다

  - **단순 무작위 배정**: 사용자가 도착할 때마다 독립적으로 반반 확률로 배정한다. 구현이 가장 간단해 트래픽이 큰 서비스의 기본값이다

  - **층화 무작위 배정**: 중요한 특성(모바일/데스크톱, 신규/기존)으로 층을 먼저 나눈 뒤 각 층 안에서 무작위 배정한다. 층마다 인원이 맞춰지므로 소규모 실험과 세그먼트별 보고에 안정적이다

- 판정 기준: 팔(비교 대상 안 하나) 당 1,000명을 넘고 세그먼트별 보고 요구가 없으면 단순 배정, 그보다 작거나 세그먼트별 결과를 보고해야 하면 층화 배정을 쓴다

- 가정이 적은 대신 실험은 비용이 비싸고 느리다. 그 비용이 어디서 얼마나 발생하는지를 숫자로 재는 것이 1부의 나머지 내용이다

한 줄 정리: 무작위 배정은 관측하지 못한 특성까지 비슷한 두 집단을 설계 단계에서 만든다. 대신 비용을 치른다.

☐ 8.1.3 표본 크기 — 실험 전에 정해야 하는 숫자

- 실험을 먼저 돌려 놓고 “언제까지 모을지”를 결과를 보면서 정하면, 좋아 보이는 시점에 멈추게 되어 결론이 부풀려진다. 그래서 **표본 크기** — 실험에 필요한 인원 — 는 시작 전에 계산해 문서에 적는다

- 계산의 입력은 세 가지다. A안의 예상 전환율, B안의 예상 전환율, 그리고 잡고 싶은 검정력(다음 절). 출력은 팔 하나에 필요한 인원이다

- 공식 대신 구조 두 가지만 기억하면 된다

  - 첫째, 필요 인원은 두 전환율의 **차이의 제곱**으로 나눠 정해지므로, **잡으려는 효과가 절반이 되면 필요 인원은 약 네 배**가 된다

  - 둘째, 전환율이 0.5에 가까울수록 결과의 흩어짐이 커져 인원이 더 필요하다

- 실습 코드(핵심 부분, 전체는 `8-1-fixed-rct-simulation.py`)

```
def required_n_per_arm(p1, p2, alpha=0.05, power=0.80):
    p_bar = (p1 + p2) / 2
    z_a = stats.norm.ppf(1 - alpha / 2)      # 1.960: 위양성 허용선(α=0.05)
    z_b = stats.norm.ppf(power)              # 0.842: 검정력 0.80
    term = (z_a * np.sqrt(2 * p_bar * (1 - p_bar))
            + z_b * np.sqrt(p1 * (1 - p1) + p2 * (1 - p2)))
    return term ** 2 / abs(p2 - p1) ** 2     # 분모가 효과 크기의 제곱
```

○ 실행 결과(`results/8-1-fixed-rct-simulation.log` 발췌)

```
  시나리오                      참 효과      필요 n/팔      필요 총원      실제 n/팔        과부족
  시나리오 1 (소효과)           +0.05        1534       3068        100      -1434
  시나리오 2 (중효과)           +0.15         173        346        100        -73
  시나리오 3 (대효과)           +0.30          42         84        100        +58
  시나리오 4 (무효과)           +0.00          무한         무한         100          -

  [손계산 확인] 시나리오 3 (0.40 → 0.70)
    n/팔 = (1.3790 + 0.5646)^2 / 0.0900 = 41.97
    statsmodels 교차 확인: 41.79
```

– 해석: 시나리오 1은 팔당 1,534명이 필요한데 100명만 있다. **필요 인원을 확보하지 못하면 실험을 시작하지 않는다** — 이 조건에서 돌리면 효과가 있어도 찾지 못한다. 반대로 시나리오 3은 42명이면 충분하므로 팔당 58명이 남고, 그 인원의 절반은 열등한 안에 불필요하게 노출된다

– 시나리오 4의 “무한”은 계산 실패가 아니다. 참 효과가 0이면 나누는 값이 0이 되므로, 아무리 모아도 “차이가 있다”를 확인할 수 없다. 설계 단계에서 잡고 싶은 최소 효과 크기를 먼저 정해야 하는 이유다

– 손계산 확인 줄에서, 스크립트가 statsmodels 라이브러리로 교차 확인한 값(41.79)과 공식 값(41.97)이 소수점 근처에서만 갈린다

한 줄 정리: 표본 크기는 실험 전에 정해 문서에 적는다. 잡으려는 효과가 절반이면 인원은 네 배다.

□ 8.1.4 검정력 — 지금 인원으로 무엇을 잡을 수 있는가

○ 현실에서는 인원이 먼저 정해져 있는 경우가 많다. 트래픽이 정해져 있거나 실험 기간이 2주로 묶여 있다면, 질문은 “몇 명이 필요한가”가 아니라 “지금 인원으로 어느 크기의 효과까지 잡을 수 있는가”로 바뀐다

○ 그 답이 **검정력(power)** 이다. 검정력 0.80은 “참 효과가 정말 그 크기라면, 같은 실험을 100번 돌렸을 때 80번은 유의하다는 결론에 도달한다”는 뜻이고, 뒤집으면 20번은 효과가 있는데도 놓친다

○ 실습 코드(핵심 부분)

```
def achieved_power(p1, p2, n_per_arm, alpha=0.05):
    delta = abs(p2 - p1)
    p_bar = (p1 + p2) / 2
    z_a = stats.norm.ppf(1 - alpha / 2)
    sd_null = np.sqrt(2 * p_bar * (1 - p_bar))
    sd_alt = np.sqrt(p1 * (1 - p1) + p2 * (1 - p2))
    return stats.norm.cdf((delta * np.sqrt(n_per_arm) - z_a * sd_null) / sd_alt)
```

○ 실행 결과(로그 발췌 — 각 팔 100명일 때)

```
  시나리오                    참 효과      검정력                    판정
  시나리오 1 (소효과)          +0.05    0.106        부족 (효과를 놓칠 확률이 큼)
  시나리오 2 (중효과)          +0.15    0.566        부족 (효과를 놓칠 확률이 큼)
  시나리오 3 (대효과)          +0.30    0.992                      충분
  시나리오 4 (무효과)          +0.00    0.050      효과 없음 (이 값은 1종 오류율)
```

– 해석: 팔당 100명으로 +0.05 효과를 잡을 확률은 0.106 — 열 번 중 아홉 번은 놓친다. 검정력 0.106짜리 실험에서 “유의하지 않았다”는 결과는 효과가 없다는 근거가 아니라, 실험이 효과를 볼 수 없었다는 사실만 알려 준다

– 시나리오 4의 0.050은 해석이 다르다. 참 효과가 0일 때 “유의하다”고 나올 확률은 검정력이 아니라 위양성 확률이며, 설계에서 정한 **유의수준** α=0.05와 같게 나왔으므로 검정은 설계대로 작동하고 있다

○ 판정 눈금: **검정력 0.80 미만이면 실험을 그대로 진행하지 않는다.** 선택지는 세 가지 — 표본을 늘리거나, 잡으려는 최소 효과 크기를 키우거나, 흩어짐이 작은 대체 지표로 1차 지표를 바꾼다. 셋 다 불가능하면 실험을 접는다

○ 판정 연습(가상의 수치다): 세 실험 계획의 검정력이 각각 0.92, 0.61, 0.35라면?

– 0.92는 그대로 진행한다. 0.61과 0.35는 진행하지 않고, 표본·효과 크기·지표 중 하나를 바꾼 뒤 다시 계산한다

한 줄 정리: 검정력 0.80이 실험 시작의 관문이다. 관문을 못 넘긴 실험의 “유의하지 않음”은 정보가 아니다.

□ 8.1.5 네 시나리오를 실제로 돌려 결과 읽기

○ 앞 두 절은 실험 전의 계산이었다. 이제 실제 데이터(`8-1-fixed-rct.csv`, 시나리오당 각 팔 100명씩 총 800명)로 돌린 결과를 읽으면서, 계산이 예고한 대로 실험이 흘러가는지 확인한다

○ 실습 코드(핵심 부분)

```
d = df[df['scenario'] == 'scenario3']
y_c = d[d['treatment'] == 0]['outcome'].values
y_t = d[d['treatment'] == 1]['outcome'].values

ate = y_t.mean() - y_c.mean()                                  # 관측 효과
_, p_value = stats.ttest_ind(y_t, y_c, equal_var=False)   # 등분산 가정하지 않는 t검정
print(ate, p_value)                                            # 0.260  0.0002
```

○ 실행 결과(로그 발췌)

```
시나리오 1 (소효과)  관측 효과 +0.060  95% CI [-0.076, +0.196]  p=0.3890  유의하지 않음
시나리오 2 (중효과)  관측 효과 +0.100  95% CI [-0.039, +0.239]  p=0.1589  유의하지 않음
시나리오 3 (대효과)  관측 효과 +0.260  95% CI [+0.126, +0.394]  p=0.0002  유의함
시나리오 4 (무효과)  관측 효과 +0.160  95% CI [+0.029, +0.291]  p=0.0176  유의함
```

- 해석: 시나리오 1·2는 참 효과가 있는데도 유의하지 않다. 검정력 0.106과 0.566이 예고한 그대로이며, “효과가 없다”가 아니라 “이 표본으로는 판단할 수 없다”로 보고한다
- 시나리오 3은 p=0.0002로 유의하고 **신뢰구간** — 참값이 들어 있을 법하다고 계산한 범위 — 도 0을 벗어난다. 다만 필요 인원 42명을 훌쩍 넘긴 초과 모집이었다
- 시나리오 4가 이 실습의 핵심이다. 두 안의 참 전환율이 똑같이 0.40인데 p=0.018로 유의하게 나왔다. 위양성(1종 오류)이며, 관측 효과 +0.160은 순전히 표본 변동이다. 유의수준 5%란 이런 실험이 20번에 1번꼴로 나오는 것을 허용한다는 뜻이다

○ 판정 기준: 유의한 결과를 받으면 세 가지를 순서대로 확인한다 — ① 사전에 정한 표본에 도달했는가 ② 중간에 여러 번 들여다보고 멈춘 것은 아닌가 ③ 효과 크기가 사업적으로 의미 있는 크기인가. 하나라도 걸리면 결론을 유보하고 재현 실험을 설계한다

○ 직접 해 보기: practice/chapter08/code 폴더에서 `python 8-1-fixed-rct-simulation.py` 실행 → 1단계 출력에서 위 네 줄이 재현되는지 확인한다

한 줄 정리: 검정력이 예고한 대로 실험은 흘러간다. 그리고 효과가 0이어도 20번에 1번은 유의하게 나온다.

□ 8.1.6~8.1.7 실험 위생, 그리고 고정 배분이 치르는 세 가지 비용

○ 알고리즘보다 기본 점검이 실험의 성패를 더 자주 가른다. 특히 자주 터지는 문제가 **표본 비율 불일치(SRM)** — 50:50으로 배정하도록 설계했는데 실제 노출 기록이 48:52처럼 어긋난 경우다

- 수백만 명 규모에서 이 정도 차이는 우연으로 설명되지 않으므로, 배정 코드나 기록 파이프라인 어딘가가 깨졌다는 신호다. 예를 들어 봇 트래픽이 한쪽 팔에서만 걸러졌다면 두 팔의 사용자 구성 자체가 달라지고, 그 차이가 처치효과로 잡힌다
- 판정 기준: 배정 비율 검정에서 $p < 0.001$이면 효과 추정치가 아무리 좋아 보여도 결과를 읽지 않고, 파이프라인을 고친 뒤 실험을 다시 돌린다

○ 실험 설계 문서에 미리 적어 둘 항목 네 가지

- **1차 지표 하나**: 지표를 여러 개 두고 그중 유의한 것을 보고하면 위양성 확률이 지표 개수만큼 올라간다
- **가드레일 지표**: 전환율이 올라도 이탈률·불만 접수·오류율이 나빠지면 배포하지 않는다는 조건
- **분석 단위**: 사용자 단위인지 세션 단위인지. 한 사용자가 여러 세션을 만들면 세션 단위 분석은 표본 수를 부풀린다
- **최소 실행 기간**: 요일 효과가 있는 서비스는 온전한 한 주 단위로 맞춘다

○ 여기까지의 계산으로 고정 배분의 비용 세 가지를 지목할 수 있다

- 첫째, 기회비용. 실험 내내 절반의 트래픽이 열등한 안에 배정된다. 이 손실을 **후회(regret)** 라고 부르며 2부에서 정확히 계산한다
- 둘째, 표본의 비효율. 효과 크기를 미리 모르므로 보수적으로 크게 잡게 된다. 시나리오 3의 초과 모집(팔당 58명)이 그 예다
- 셋째, 늦은 결론. 2주짜리 실험이 끝날 무렵 시즌이 바뀌었다면 결론을 적용할 대상이 남아 있지 않다

○ 세 비용 모두 배분 비율과 종료 시점을 실험 시작 전에 고정했기 때문에 발생한다. 8.2는 종료 시점을, 2부는 배분 비율을 실험 도중에 바꾸는 설계를 다룬다

한 줄 정리: SRM이 뜨면 결과를 읽지 않는다. 고정 배분의 비용은 모두 “미리 고정”에서 나온다.

□ 8.2.1 중간에 들여다보면 생기는 문제

○ 앞 절에 남은 질문: 효과가 확실해 보이면 그때 멈추면 되지 않는가. 이 방식을 그대로 쓰면, 참 효과가 0인데도 유의하다고 판정할 확률이 설계한 5%보다 훨씬 커진다. 얼마나 커지는지를 먼저 잰다

○ 계산 방법: 두 안의 효과가 정말 같다고 두고, 실험을 K번 나누어 **중간 분석** — 실험이 끝나기 전에 결과를 미리 들여다보는 일 — 을 하면서 매번 $p < 0.05$로 판정한다. 한 번이라도 유의하면 멈추고 "효과가 있다"고 선언한다. 이 절차를 20만 번 반복해 잘못 선언한 비율을 센다

○ 실습 코드(핵심 부분, 전체는 `8-2-group-sequential-design.py`)

```
def z_paths(k, n_path=200_000, seed=0):
    rng = np.random.default_rng(seed)
    steps = rng.standard_normal((n_path, k))   # 단계마다 새로 들어온 정보
    cum = np.cumsum(steps, axis=1)             # 누적합
    info = np.sqrt(np.arange(1, k + 1))        # 누적 정보량의 제곱근
    return cum / info                          # 누적 검정 통계량
```

○ 실행 결과(`results/8-2-group-sequential-design.log` 발췌)

```
들여다본 횟수 K        1종 오류        명목 0.05의 몇 배
          1      0.0498                1.00
          2      0.0825                1.65
          3      0.1062                2.12
          5      0.1409                2.82
         10      0.1911                3.82
         20      0.2484                4.97
```

– 해석: 한 번만 판정하면 0.0498로 설계값 그대로다. 그런데 다섯 번 들여다보면 0.1409 — 효과가 전혀 없는 실험 일곱 개 중 하나에서 "유의하다"는 결론이 나온다. 스무 번이면 네 개 중 하나다

○ 구조는 주사위 던지기와 같다. 한 번 던지기가 중간 분석 한 번에, 6이 나오는 사건이 $p < 0.05$ 관측에, "6이 나오면 멈추고 이겼다고 선언"이 "유의하면 멈추고 효과가 있다고 선언"에 대응한다. 여러 번 던질수록 한 번쯤 6이 나올 확률이 오르듯, 여러 번 볼수록 한 번쯤 유의해 보일 확률이 오른다

– 다만 주사위는 매번 독립이지만, 중간 분석은 앞 단계 데이터를 포함한 누적 통계량을 쓰므로 단계끼리 서로 닮아 있다. 그래서 상승폭이 단순 곱셈($5 \times 0.05 = 0.25$)보다 작은 0.1409에 그친다. 위 표의 마지막 열이 K에 비례하지 않는 이유다

○ 판정 기준: 결과를 확인하는 횟수와 시점은 실험 시작 전에 정해 문서에 적는다. 정하지 않고 들여다본 뒤 유의해서 멈춘 실험의 p값은 그대로 보고할 수 없고, 재현 실험으로 다시 확인한다

한 줄 정리: 보정 없이 다섯 번 들여다보면 위양성이 5%가 아니라 14%다.

□ 8.2.2 그룹 순차 설계 — 경계를 바꿔 오류율을 되돌린다

○ 앞 절의 결론이 “중간에 보지 말라”로 끝나면, 나쁜 안에 트래픽을 계속 보내는 실험을 끝까지 끌고 가야 한다. 필요한 것은 **중간에 보되 전체 오류율은 5%로 유지하는 규칙**이며, 그것이 **그룹 순차 설계**다

○ 발상은 단순하다. 여러 번 보면 오류가 부풀므로, 매번의 판정 기준(**중단 경계**)을 원래보다 엄격하게 만들어 전체를 5%로 되돌린다. 경계의 모양을 먼저 정하고, 곱할 상수 하나를 시뮬레이션으로 찾는다

– **Pocock 경계**: 모든 단계에 같은 경계를 쓴다

– **O’Brien-Fleming(OBF) 경계**: 초기에 극도로 엄격하고 후기로 갈수록 풀린다

– 이 실습의 설계: 총 200명을 67명·133명·200명 시점에 세 번 판정한다. 계획 표본 대비 지금까지 모인 비율(0.33, 0.67, 1.00)을 **정보 분획**이라고 부른다

○ 실습 코드(핵심 부분)

```
t = np.arange(1, 4) / 3                  # 정보 분획 0.33, 0.67, 1.00
shape_pocock = np.ones(3)                # 모든 단계에서 같은 경계
shape_obf = 1 / np.sqrt(t)               # 초기 엄격, 후기 완화

c_pocock = calibrate(paths3, shape_pocock)   # 전체 오류가 0.05가 되는 상수를 찾는다
c_obf = calibrate(paths3, shape_obf)
```

○ 실행 결과(로그 발췌)

```
Pocock 상수           c = 2.2944
O'Brien-Fleming 상수  c = 2.0024  (보정 전 1.96)

   단계      정보 분획     누적 n   Pocock z   Pocock p    OBF z      OBF p
    1       0.33       67      2.294    0.02177    3.468    0.00052
    2       0.67      133      2.294    0.02177    2.452    0.01419
    3       1.00      200      2.294    0.02177    2.002    0.04525

무보정 (매번 1.96)        1종 오류 0.1161   1단계에서 중단 0.0611
Pocock               1종 오류 0.0510   1단계에서 중단 0.0213
O'Brien-Fleming      1종 오류 0.0520   1단계에서 중단 0.0004
```

– 해석: 1단계에서 OBF로 멈추려면 $p < 0.00052$여야 하고, Pocock이면 $p < 0.02177$이면 된다. Pocock이 훨씬 자주 조기 종료된다. 반대로 마지막 단계에서는 OBF가 0.04525로 관대하고 Pocock은 여전히 0.02177로 엄격하다

- 아래 세 줄은 두 안의 전환율을 모두 0.40으로 두고 2만 번 실험을 반복한 검증이다. 무보정은 0.1161로 명목의 두 배를 넘고, 보정한 두 경계는 0.0510과 0.0520으로 5% 근처로 돌아온다. 1단계 중단 확률(0.0213 대 0.0004)에서 두 경계의 성격 차이가 드러난다

○ 판정 기준: 경계를 넘으면 그 단계에서 종료하고, 못 넘으면 다음 단계까지 계속 모집한다. “거의 넘을 뻔했다”는 이유로 기준을 낮추지 않으며, 경계값은 실험 시작 전에 계산해 문서에 적는다

한 줄 정리: 매번의 기준을 엄격하게 만들면, 세 번 들여다봐도 전체 오류율은 5%로 돌아온다.

□ 8.2.3 Pocock과 O’Brien-Fleming 중 무엇을 고르는가

○ 두 경계는 같은 5%를 단계에 나누는 방식만 다르다. 조기 종료의 가치와 완주의 가치를 비교해 고른다

- **OBF를 고르는 경우**: 최종 결론의 정확도가 중요할 때. 1단계 경계가 극도로 엄격해 표본이 적은 초기의 극단값에 흔들리지 않으므로, 규제 제출용 임상시험과 정책 평가의 기본값이다

- **Pocock을 고르는 경우**: 조기 종료 자체가 큰 이득일 때. 열등한 안 노출 비용이 크거나 운영비가 높은 상황이다. 대신 마지막 단계에서도 $p < 0.02177$을 요구하므로 완주 시 유의성 확보가 더 어렵다

○ 중간 분석 횟수는 3~5회가 표준이다. 너무 잦으면 매번의 경계가 지나치게 엄격해져 조기 종료의 이점이 사라진다

○ 판정 연습(가상의 수치다): 위 경계표를 쓰는 실험의 2단계(누적 133명)에서 p=0.018이 관측되었다면?

- Pocock이면 $0.018 < 0.02177$이므로 중단하고 결론을 낸다. OBF면 $0.018 > 0.01419$이므로 멈추지 못하고 3단계까지 모집한다. 같은 데이터라도 시작 전에 고른 경계에 따라 행동이 갈린다

한 줄 정리: 완주와 최종 정확도가 중요하면 OBF, 조기 종료의 이득이 크면 Pocock이다.

□ 1부 정리

○ 지금까지 만든 것 두 가지: 네 시나리오의 요약표(참 효과·필요 표본·검정력·관측 효과·p값), 그리고 중간 분석의 중단 경계표

○ 확인한 판정 기준

- 필요 표본을 확보하지 못하면 실험을 시작하지 않는다. 검정력 0.80 미만이면 표본·효과 크기·지표 중 하나를 바꾼다

- 배정 비율이 설계와 어긋나면(SRM) 결과를 읽지 않고 파이프라인부터 고친다
- 중간에 여러 번 판정하려면 경계를 보정한다. 보정 없이 세 번 보면 1종 오류가 0.1161이고, 보정하면 0.05 근처로 돌아온다
- 경계값과 중간 분석 시점은 실험 시작 전에 문서에 적고 실행 중에 바꾸지 않는다

○ 여기까지는 배분을 반반으로 고정한 채 종료 시점만 조정했다. 2부의 질문: 열등한 안에 절반의 트래픽이 계속 흘러가는 손실 자체를 줄이려면, 배분 비율을 실험 도중에 바꾸면 되지 않는가

2부. 배분 비율을 바꾸는 설계로 손실을 줄인다

□ 2부 개요

○ 이 파트가 끝나면: 톰슨 샘플링이 기대 손실을 얼마나 줄이는지(133.33 → 3.75)와 그 대가(열등한 안의 추정 포기)를 수치로 읽고, 사후확률로 중단을 판정하는 베이지안 설계를 돌린 뒤, 새 과제에 고정 배분과 적응형 중 무엇을 쓸지 결정할 수 있다

○ 다루는 절: 8.3 ~ 8.7

□ 8.3.1~8.3.2 탐험과 활용 — 슬롯머신 문제

○ **반응 적응형 무작위배정(RAR)** 은 실험 도중 관측되는 결과에 따라 배정 확률을 바꾸는 설계다. 그룹 순차 설계가 종료 시점만 바꿨다면, RAR은 배분 비율 자체를 바꾼다

○ 여기서 두 목표가 충돌한다

- **탐험(exploration)**: 모든 안을 충분히 시도해 어느 것이 좋은지 알아내는 일
- **활용(exploitation)**: 지금까지 좋아 보이는 안을 많이 써서 성과를 올리는 일
- 탐험에만 매달리면 열등한 안에 계속 트래픽을 보내고, 활용에만 매달리면 초기 몇 번의 운으로 잘못 고른 안에 고착된다

○ 이 문제를 정식화한 틀이 **멀티 암드 밴딧(MAB)** — 슬롯머신 여러 대 문제다. 이름 자체가 비유이므로 대응을 짚어 둔다

카지노 상황	실험 상황
슬롯머신 한 대	비교 대상 안 하나

카지노 상황	실험 상황
레버를 한 번 당김	사용자 한 명에게 그 안을 노출
기계마다 숨어 있는 당첨 확률	각 안의 참 전환율
최고 기계만 당겼을 때 수익과 실제 수익의 차이	누적 후회(regret)

○ 비유가 깨지는 지점 두 가지를 적용 전에 확인한다

– 슬롯머신의 당첨 확률은 고정이지만, 서비스의 전환율은 요일·시즌·유입 경로에 따라 움직인다. 참 전환율이 변하면 과거 데이터 기반 배분이 틀린 안을 밀어줄 수 있다

– 카지노의 레버 당기기는 서로 독립이지만, 서비스에서는 같은 사용자가 여러 안을 순서대로 보며 먼저 본 안이 뒤의 반응에 영향을 줄 수 있다

한 줄 정리: 배분을 바꾸는 설계는 “알아내기”와 “벌기”의 교환 문제다.

□ 8.3.3 세 알고리즘이 다음 배정을 정하는 방식

○ 세 알고리즘 모두 “다음 한 명을 어느 안에 보낼 것인가”에 답한다. 기준이 다르다

– **Epsilon-Greedy**: 정해 둔 작은 확률(예: 10%)로는 아무 안이나 무작위로 고르고, 나머지는 지금까지 평균 전환율이 가장 높은 안을 고른다. 가장 간단하지만 그 확률을 사람이 정해야 하고, 최적 안을 찾은 뒤에도 계속 10%를 낭비한다

– **UCB**: 각 안의 평균 전환율에 “시도가 적을수록 커지는 보너스”를 더해 가장 큰 안을 고른다. 데이터가 적은 안에 자동으로 기회가 돌아가고, 사람이 정할 값이 없다

– **톰슨 샘플링(Thompson Sampling)**: 각 안의 전환율에 대한 확률 분포를 유지하고, 매번 분포에서 값을 하나씩 뽑아 가장 큰 값이 나온 안을 고른다

○ 톰슨 샘플링의 절차는 네 단계다

– ① 안마다 성공 횟수와 실패 횟수를 센다 ② 그 두 숫자로 “전환율이 얼마쯤일까”의 분포(Beta 분포)를 만든다 ③ 각 안의 분포에서 난수를 하나씩 뽑는다 ④ 가장 큰 난수가 나온 안에 다음 사용자를 보내고, 반응을 관측해 ①로 돌아간다

○ 탐험이 저절로 조절되는 까닭은 분포의 폭에 있다. 시도가 적은 안은 분포가 넓게 퍼져 있어 가끔 큰 값이 뽑혀 기회를 얻고, 시도가 쌓이면 분포가 좁아져 참값 근처의 값만 뽑힌다. 탐험 비율을 사람이 정할 필요가 없다는 점이 Epsilon-Greedy와의 결정적 차이다

한 줄 정리: 톰슨 샘플링은 “성공·실패를 세고, 분포에서 뽑고, 큰 쪽에 보낸다”의 반복이다.

□ 8.3.4 톰슨 샘플링을 돌려 결과 읽기

○ 데이터는 `8-3-thompson-sampling.csv` — 참 전환율이 각각 0.30, 0.55, 0.40인 세 안(Arm)에 톰슨 샘플링을 1,000회 돌린 기록이다

○ 성과의 자는 **기대 후회**다. 안마다 "그 안에 보낸 인원 × 최적 안과의 전환율 차이"를 구해 더한다

– 이번 실행에서는 6명×0.25 + 979명×0.00 + 15명×0.15 = 3.75. 배분 인원만으로 정해지므로 실행할 때의 운에 흔들리지 않는다

○ 실습 코드(핵심 부분, 전체는 `8-3-thompson-sampling.py`)

```
counts = np.bincount(df['arm_selected'], minlength=3)     # 팔별 배분 인원
true_probs = np.array([0.30, 0.55, 0.40])
p_star = true_probs.max()

def expected_regret(counts):
    return float(np.sum(counts * (p_star - true_probs)))  # 인원 x 전환율 격차의 합
```

○ 실행 결과(`results/8-3-thompson-sampling.log` 발췌)

```
    Arm       참 전환율       배분 인원       배분 비율      성공       사후평균       참값과 차이
      0        0.30           6        0.6%       0      0.125       -0.175
      1        0.55         979       97.9%     569      0.581       +0.031
      2        0.40          15        1.5%       3      0.235       -0.165

      시점 t      Arm 0      Arm 1      Arm 2
        50       8.0%      76.0%      16.0%
       200       3.0%      91.0%       6.0%
      1000       0.6%      97.9%       1.5%

                  배분 방식 최적팔 배분비율  기대 손실 실현 보상
               고정 1:1:1    33.3% 133.33      -
Epsilon-Greedy (ε=0.1)    73.1%  43.35   524
                톰슨 샘플링    97.9%   3.75   572
```

– 해석: 50회 시점에 이미 최적 안(Arm 1)에 76%가 가 있고, 1,000회에는 97.9%까지 오른다. 고정 배분이면 이 값이 끝까지 33.3%다. 기대 손실은 133.33에서 3.75로 97.2% 줄었으며, 열등한 안을 겪은 사용자가 약 130명분 줄었다는 뜻이다. 전환 하나의 가치가 정해져 있다면 금액으로 환산할 수 있다

○ 판정 기준: 도입 여부는 기대 손실 감소분을 전환 가치로 환산해 도입 비용(실시간 파이프라인, 운영 복잡도)과 비교해 정한다. 총 시행이 수백 회 수준이면 배분이 쏠리기 전에 실험이 끝나므로 이득이 크지 않다

한 줄 정리: 톰슨 샘플링은 최적 안에 97.9%를 보내 기대 손실을 97.2% 줄였다.

□ 8.3.5 적응형 배분이 포기하는 것

○ 앞 표만 보면 적응형이 모든 면에서 낫다는 결론이 나온다. 같은 로그의 5단계 출력에 그 대가가 있다

○ 실행 결과(로그 발췌)

```
Arm      배분  인원      사후평균      참값      절대오차              95% 신용구간
  0         6     0.125    0.30     0.175        [0.004, 0.410]
  1       979     0.581    0.55     0.031        [0.550, 0.612]
  2        15     0.235    0.40     0.165        [0.073, 0.456]
```

– 해석: Arm 1의 추정은 정확하다(오차 0.031, 구간 폭 0.062). 그러나 6명만 배정된 Arm 0의 추정값 0.125는 참값 0.30에서 0.175나 떨어져 있고, 구간 폭은 0.4를 넘는다. 적응형 배분은 **최적 안을 찾고 손실을 줄이는 데는 강하지만, 열등한 안의 효과를 정확히 재는 일은 포기한다**

○ 판정 기준: 배분 인원 30명 미만인 안의 추정치는 보고서에 싣지 않는다. 과제의 목적이 효과 크기 추정이면 고정 배분을, 운영 성과 개선이면 적응형을 쓴다. 두 목적을 한 실험으로 동시에 달성하려는 설계는 대개 둘 다 놓친다

한 줄 정리: 손실 감소의 대가는 열등한 안의 추정 포기다. 목적이 추정이면 적응형을 쓰지 않는다.

□ 8.4.1~8.4.2 사후확률과 Beta 갱신 — 판정 문장을 바꾼다

○ 그룹 순차 설계는 p값으로 중단을 판정한다. p값은 “효과가 없다고 가정할 때 이만큼 극단적인 결과가 나올 확률”이라 조건절을 붙여 읽어야 하는데, 실무 회의에서는 거의 항상 “B안이 나을 확률 95%”로 잘못 옮겨진다

○ **베이지안 적응형 설계**는 그 잘못된 번역을 정식 계산으로 만든다. **사후확률** — 지금까지의 데이터를 본 후에 “B안의 전환율이 A안보다 높을” 확률 — 은 문자 그대로 조건절 없이 읽는다

○ 계산 절차는 세 단계다

– ① 각 안의 전환율에 대해 **사전분포** — 데이터를 보기 전의 믿음 — 를 “0과 1 사이 모든 값이 똑같이 그럴듯하다”로 놓는다(Beta(1,1))

- ② 관측이 쌓이면 성공 횟수를 앞 숫자에, 실패 횟수를 뒤 숫자에 더해 **사후분포**를 만든다. 예를 들어 25명 중 19명이 전환했으면 Beta(1+19, 1+6) = Beta(20, 7)이다
- ③ 두 안의 사후분포에서 각각 난수를 많이 뽑아 "B의 값이 A의 값보다 큰 비율"을 세면 그것이 사후확률이다

○ 실습 코드(핵심 부분, 전체는 `8-4-bayesian-adaptive.py`)

```
pa = rng_mc.beta(1 + s_c, 1 + n_c - s_c, 200_000)   # A안 사후분포에서 뽑기
pb = rng_mc.beta(1 + s_t, 1 + n_t - s_t, 200_000)   # B안 사후분포에서 뽑기
pp = float(np.mean(pb > pa))                        # P(p_B > p_A | 데이터)

# 중단 규칙: pp >= 0.99 → B안 채택·중단 / pp <= 0.05 → B안 폐기·중단 / 그 사이 → 계속
```

○ 중단 기준 0.99와 0.05는 각각 **효능 중단**과 **무용성 중단**의 문턱이다. 99% 확률로 B가 우수하면 더 볼 이유가 없고, 5% 이하로만 우수하면 계속할 이유가 없다

한 줄 정리: 사후확률은 "B안이 나을 확률"로 바로 읽는다. 중단 문턱은 위·아래 두 개다.

□ 8.4.3 세 시나리오 실행 결과 읽기

○ 시나리오마다 200명까지 모집하되, 누적 50·100·150·200명 시점에 중간 분석을 넣는다. 참 전환율은 대효과 0.40 대 0.70, 중효과 0.40 대 0.55, 무효과 0.40 대 0.40이다

○ 실행 결과(`results/8-4-bayesian-adaptive.log` 발췌)

```
[대효과]  누적 n    A안 성공/인원   B안 성공/인원   P(p_B>p_A)    결정
             50       11/25         19/25        0.9887      계속
            100       21/50         37/50        0.9993      B안 채택·중단

[무효과]     50       11/25          9/25        0.2865      계속
            100       25/50         18/50        0.0803      계속
            150       39/75         22/75        0.0023      B안 폐기·중단

시나리오    참 A    참 B  최종 n  조기 중단     이유       최종 사후확률  절약한 인원
 대효과    0.40    0.70   100      예      efficacy       0.9993        100
 중효과    0.40    0.55   200     아니오       -           0.9545          0
 무효과    0.40    0.40   150      예      futility       0.0023         50
```

- 해석: 대효과는 n=50에서 이미 0.9887이었지만 기준 0.99에 0.0013 모자라 계속 모집했고, n=100에서 0.9993으로 넘어 중단했다. "거의 넘었다"로 멈추지 않는 규칙이 여기서도 적용된다

- 중효과는 사후확률이 0.8718 → 0.9441 → 0.9632 → 0.9545로 움직이며 끝내 0.99에 닿지 못하고 200명을 완주했다. 이 결과는 “유의하지 않았다”가 아니라 “이 표본으로는 0.99 기준을 충족하지 못했다”로, 판정 유보로 적는다

- 세 실험을 고정 설계로 돌렸다면 600명이 필요했으나 실제로는 450명을 썼다. 150명을 아꼈다

○ 직접 해 보기: `python 8-4-bayesian-adaptive.py` 실행 → 2단계 출력에서 시나리오별 사후확률 궤적이 위와 같은지 확인한다

한 줄 정리: 효과가 크면 일찍 멈추고, 없으면 무용성으로 멈추고, 애매하면 완주 후 판정을 유보한다.

□ 8.4.4 중단 기준을 어디에 두는가

○ 0.99라는 문턱은 자연법칙이 아니라 설계 선택이다. 로그 4단계에, 효능 기준을 0.95로 낮추면 무엇이 달라지는지가 있다

- 대효과: n=100 → n=50에서 중단(50명 더 절약). 중효과: 완주 → n=150에서 중단. 무효과: 그대로 중단 없음(무용성 기준이 따로 작동)

- 기준을 낮추면 실험이 짧아지는 대신 위양성 위험이 함께 오른다

○ 첫 중간 분석 시점도 설계 선택이다. 무효과 시나리오의 첫 시점(각 군 25명) 값 0.2865가 n=100에서 0.0803까지 떨어진 궤적을 보면, 초기 값이 얼마나 흔들리는지 드러난다. 첫 중간 분석은 각 군 최소 50명이 모인 뒤에 배치한다

○ 판정 기준: 중단 기준과 중간 분석 시점은 실험 시작 전에 정하고 실행 중에 바꾸지 않는다. 규제 제출·외부 보고가 걸린 실험이면 0.99, 내부 운영 의사결정이면 0.95도 쓸 수 있다

○ 판정 연습(가상의 수치다): 중간 분석에서 사후확률이 0.97, 0.031로 나온 두 실험이 있다면?

- 기준이 0.99/0.05일 때, 0.97은 두 문턱 사이이므로 계속 모집하고, 0.031은 0.05 이하이므로 B안을 접고 중단한다. 기준이 0.95였다면 첫 실험은 채택·중단이다 — 기준을 언제 정했는지가 결론을 가른다

한 줄 정리: 문턱과 분석 시점은 실험 전에 정한 값만 유효하다. 결과를 본 뒤 바꾼 기준은 기준이 아니다.

□ 8.5.1~8.5.2 사례 — 콘텐츠 5종을 두고 배분을 바꾸는 서비스 실험

○ 선택지가 둘일 때와 다섯일 때는 적응형 배분의 움직임이 다르다. 가상의 불면증 케어 앱에서 콘텐츠 5종 중 수면 개선 반응이 좋은 것을 찾는 상황이다. 데이터는 `8-5-korean-dtx.csv`(800명), 참 성공률은 마음챙김 0.52, 수면 개선 0.48, 인지 재구성 0.45, 호흡 훈련 0.38, 운동 독려 0.35로 설정되어 있다

– 디지털 치료기기 영역에서는 효과가 낮은 콘텐츠에 참가자를 오래 노출하는 일 자체가 부담이므로, 배분을 바꿔 노출을 줄이는 설계가 실질적 의미를 갖는다

○ 실습 코드(핵심 부분, 전체는 `8-5-korean-dtx-case.py`)

```
for participant in range(n_total):
    samples = np.random.beta(alpha, beta)          # 콘텐츠 5개의 사후분포에서 각각 뽑기
    selected = np.argmax(samples)                  # 가장 큰 값이 나온 콘텐츠 선택
    success = np.random.binomial(1, true_rates[selected])
    if success:
        alpha[selected] += 1                       # 성공이면 앞 숫자에 1
    else:
        beta[selected] += 1                        # 실패면 뒤 숫자에 1
```

○ 실행 결과(`results/8-5-korean-dtx-case.log` 발췌)

```
  콘텐츠              참 성공률      n=200      n=400      n=600      n=800
  마음챙김              0.52      25.5%      20.2%      27.3%      44.2%
  수면 개선             0.48      48.0%      42.2%      34.8%      26.9%
  호흡 훈련             0.38       6.0%       6.8%       5.0%       3.8%

  적응형 배분, 기대 성공자       : 380.2명
  고정 배분(1:1:1:1:1) 기대 성공자: 348.8명
  전부 최고 콘텐츠일 때 기대     : 416.0명
  기대값끼리 비교(운의 영향 없음): 348.8 → 380.2명 (+31.4명, +9.0%)
  실현값과 고정 배분 기대값 비교 : 348.8 → 382명 (+33.2명, +9.5%)
```

– 해석: n=200 시점에는 2위인 수면 개선이 48.0%로 가장 많이 배정되었고, 참 성공률 1위인 마음챙김은 800명 시점에야 44.2%로 올라섰다. 참 성공률 차이가 0.52 대 0.48로 작으면 갈라내는 데 그만큼 많은 관측이 필요해, 선택지가 다섯이면 수렴이 훨씬 느리다

– 성과 개선은 +9.0%와 +9.5% 두 값이 있는데, **보고서에는 9.0%를 쓴다.** 9.0%는 배분 인원만으로 정해지는 기댓값끼리의 비교이고, 9.5%는 이번 실행의 운이 섞인 실현값과의 비교라 다시 돌리면 재현되지 않는다

- 참 성공률이 가장 낮은 운동 독려는 고정 배분이면 160명이 노출될 자리에 89명만 배정되었다. 71명이 더 나은 콘텐츠를 받았다는 뜻이며, 전환 지표와 별개로 보고할 만한 결과다

한 줄 정리: 다섯 안이면 수렴이 느려도 기대 성공자는 +9.0% 늘었다. 보고는 실현값이 아니라 기댓값 비교로 한다.

☐ 8.5.3~8.5.4 순위의 함정, 그리고 가드레일

○ 로그 5단계는 두 순위를 나란히 놓는다

```
  참 성공률 순위 : 마음챙김(0.52) > 수면 개선(0.48) > 인지 재구성(0.45) > 호흡 훈련(0.38) > 운동 독려(0.35)
  배분 인원 순위 : 마음챙김(354명) > 수면 개선(215명) > 인지 재구성(112명) > 운동 독려(89명) > 호흡 훈련(30명)
```

- 해석: 1~3위는 맞지만 4위와 5위가 뒤집혔다. 참 성공률 0.38인 호흡 훈련이 0.35인 운동 독려보다 적게 배정되었다 — 차이 0.03은 800명으로 갈라지지 않았다

○ 판정 기준: 배분 인원 순위를 콘텐츠 효과 순위로 읽지 않는다. 순위를 보고해야 하는 과제라면 추정치와 구간을 보고, 구간이 겹치는 콘텐츠끼리는 “순위를 가릴 수 없음”으로 적는다. 이번 실행에서 호흡 훈련의 구간 [0.142, 0.446]과 운동 독려의 구간 [0.288, 0.486]은 크게 겹친다

○ 헬스케어·금융처럼 실패의 비용이 큰 영역에서는 설계 문서에 가드레일을 함께 적는다

- 악화 지표(수면 지표가 나빠진 참가자 비율)에 상한을 두고, 넘으면 그 콘텐츠 배분을 중단한다
- 최소 배분 하한(예: 각 콘텐츠 5%)을 두어, 초기 몇 번의 운으로 좋은 콘텐츠가 영구히 배제되는 일을 막는다
- 특정 연령대·성별에서만 성과가 나빠지는지 중간 분석 때마다 확인하고, 채택 규칙은 “효과가 좋고 **동시에** 안전 지표가 나빠지지 않아야 한다”의 결합으로 적는다

한 줄 정리: 배분 순위는 효과 순위가 아니다. 구간이 겹치면 순위를 가리지 않는다.

☐ 8.6~8.7 확장 — 문맥적 밴딧과 하이브리드 설계

○ 지금까지의 알고리즘은 모든 사용자에게 같은 기준으로 안을 고른다. 실무의 확장은 여기서 한 걸음 더 나간다

- **문맥적 밴딧**: 사용자가 도착할 때 관측되는 맥락(디바이스, 유입 경로, 시간대)을 함께 보고 안을 고른다. 목표가 "전체에서 가장 좋은 하나"에서 "각 맥락에서 가장 좋은 하나"로 바뀐다
- **강화학습**: 지금의 선택이 미래 보상에 미치는 영향까지 계산에 넣는다. 당장의 할인 쿠폰이 이번 달 전환율을 올리지만 다음 분기 이탈률을 높인다면, 단일 시점 밴딧은 이를 보지 못한다
- **제약 조건이 붙은 밴딧**: 8.5.4의 가드레일을 알고리즘 안에 제약식으로 넣어, 공정성·안전성 요건을 지키면서 최적화한다

○ 실무에서 자주 쓰이는 조합 세 가지

- **앙상블 톰슨 샘플링**: 여러 밴딧 모델을 병렬로 돌리고 투표로 정한다. 단일 모델이 초기 데이터의 특이한 패턴에 고착되는 위험을 줄이는 대신 운영 복잡도가 오른다
- **단계 전환형 하이브리드**: 앞 국면은 고정 배분으로 각 안의 효과를 구간과 함께 추정하고, 뒤 국면은 적응형으로 전환해 손실을 줄인다. 8.3.5의 "추정과 손실 감소를 동시에 못 얻는다"는 문제를 시간으로 나누어 푼다
- **그룹 순차 설계와의 결합**: 배분은 적응형으로 두되 종료 판정에는 보정한 경계를 쓴다. 배분이 바뀌면 정보량 계산이 복잡해지므로 경계를 공식이 아니라 시뮬레이션으로 잡는다

한 줄 정리: 확장의 방향은 "맥락을 본다, 미래를 본다, 제약을 지킨다"의 셋이고, 실무는 설계를 섞어 쓴다.

□ 결과를 읽는 순서

○ 이 장의 산출물은 다음 순서로 읽는다

- 1순위 **배정 비율(SRM)**: 설계와 어긋나면 뒤의 숫자는 읽지 않고 파이프라인부터 고친다
- 2순위 **표본과 검정력**: 필요 표본에 도달했는지, 검정력 0.80을 넘겼는지 본다. 못 넘긴 실험의 "유의하지 않음"은 결론이 아니다
- 3순위 **중단 절차의 적법성**: 중간에 본 횟수·시점이 사전 계획과 같은지, 경계 보정을 했는지 확인한다. 계획은 3회인데 5회 들여다봤다면 그 경계는 무효다
- 4순위 **효과와 구간**: 여기까지 통과한 뒤에야 효과 크기를 읽는다. 적응형 실험이라면 배분 30명 미만인 안의 추정치는 건너뛴다
- 5순위 **성과 지표의 종류**: 손실·개선율은 실현값이 아니라 기댓값 기준 수치를 채택한다

□ 실무 적용 점검표

○ 목적을 하나로 정했는가 — 효과 크기를 구간과 함께 보고하는 것인지, 실험 기간의 누적 성과를 올리는 것인지. 둘 다라고 답하면 설계가 흔들린다

○ 1차 지표 하나와 가드레일 지표를 실험 전에 문서에 적었는가

○ 필요 표본과 검정력을 계산했는가 — 검정력 0.80 미만이면 시작하지 않는다

○ 중간 분석 횟수(3~5회)와 시점을 정했는가 — 첫 시점은 각 군 50명 이후, 정한 뒤에는 바꾸지 않는다

○ 설계를 목적에 맞게 골랐는가 — 효과 추정이면 고정 배분 + 그룹 순차, 운영 성과면 적응형, 둘 다면 국면을 나누는 하이브리드

○ 배정 비율을 매일 점검하는가 — SRM이 뜨면 결과를 읽지 않는다

○ 종료 후 보고 항목을 확인했는가 — 기댓값 기준 성과 개선, 배분이 충분한 안의 추정치, 가드레일 지표의 변화

□ 핵심 정리

설계	무엇을 바꾸는가	강한 점	포기하는 것
고정 배분 A/B	아무것도 바꾸지 않음	모든 안의 효과를 같은 정확도로 추정	실험 기간의 누적 손실
그룹 순차 설계	종료 시점	오류율을 지키면서 조기 종료	경계 설계와 사전 문서화 부담
반응 적응형 배분	배분 비율	누적 손실을 크게 줄임	열등한 안의 효과 추정
베이지안 적응형	종료 시점과 판정 문장	확률로 바로 읽는 결론	사전분포·기준값 선택의 책임

○ 이 장의 실측 수치를 한 줄씩 옮기면 이렇다

– 팔당 100명으로 +0.05 효과를 잡을 확률은 0.106이고, 효과가 0인데도 p=0.018이 나오는 실험이 있었다

– 보정 없이 다섯 번 들여다보면 1종 오류가 0.1409로 오르고, 보정한 경계(Pocock 0.0510, OBF 0.0520)는 0.05 근처로 되돌린다

– 톰슨 샘플링은 1,000회 시행에서 최적 안에 97.9%를 보내 기대 손실을 133.33에서 3.75로 줄였고, 그 대가로 6명만 배정된 안의 추정 오차가 0.175였다

- 베이지안 적응형은 세 실험에서 600명 대신 450명을 썼고, 콘텐츠 5종 사례의 기댓값 기준 개선은 +9.0%였다

○ 이 장의 판정 규칙 세 가지

- **실험 전에 정한 숫자만 유효하다.** 표본 크기·중간 분석 시점·중단 경계·사후확률 문턱 모두, 결과를 본 뒤 바꾸면 무효다
- **적응형 배분의 산출물에서 배분이 적은 안의 추정치와 배분 순위는 보고하지 않는다.** 손실 감소는 얻고 추정은 내준 설계이기 때문이다
- **성과는 기댓값 기준으로 보고한다.** 실현값에는 이번 실행의 운이 섞여 있어 재현되지 않는다